IP Centrul de Excelență în Informatică
și Tehnologii Informaționale (CEITI)

Catedra de Informatică I

Studiul Individual Nr. 1

Disciplina: **Programarea vizuală**

Tema:

Dezvoltarea jocului desktop
Zombie Survivors în C# Windows Forms -
Proiectarea interfeței grafice
și implementarea funcționalităților

Elev: Bragari Eduard

Grupa: P-2432

Profesor: Diana Nebunealia

Chișinău, 2026

Cuprins

1. Documentare teoretică 4
1.1 Ce este programarea vizuală? 4
1.2 Ce este Windows Forms? 4
1.3 Limbajul C# și platforma .NET 4
1.4 Cerințe funcționale 5
1.5 Cerințe non-funcționale 5
1.6 Tabel comparativ 5

2. Prezentarea aplicației 6
2.1 Descrierea jocului 6
2.2 Funcționalități principale 6
2.3 Tehnologii utilizate 6
2.4 Captură generală a jocului 6

3. Structura proiectului 7
3.1 Organizarea fișierelor 7
3.2 Rolul formularului Form1 7
3.3 Legătura dintre componente 7

4. Realizarea practică a interfeței 8
4.1 Crearea proiectului Windows Forms 8
4.2 Utilizarea Toolbox-ului 9
4.3 Form Designer și Properties 10
4.4 Crearea meniului principal 12
4.5 Crearea barei HP 13
4.6 Bara XP și nivelul 14
4.7 Interfața pentru valuri 15
4.8 Interfața Level Up 17
4.9 Pause Menu 20
4.10 Settings 21
4.11 Game Over și Respawn 22
4.12 Debug Menu 23

4.13 Legătura dintre UI și cod 24

5. Explicarea codului jocului 27

5.1 Mișcarea jucătorului 27

5.2 Sistemul de shooting 28

5.3 Inamicii 29

5.4 Sistemul de valuri 29

5.5 XP și Level Up 30

5.6 Upgrade-urile 30

5.7 Sistemul HP și bandajele 30

5.8 Randarea grafică 30

6. Reflecție personală 31

7. Git și GitHub 31

7.1 Ce sunt Git și GitHub? 31

7.2 Utilizarea Git în proiect 32

7.3 Repository-ul proiectului 32

8. README și alte comenzi Git 32

8.1 Ce este README? 32

8.2 Comenzi Git utile 32

9. Concluzie 33

10. Bibliografie 34

1. Documentare teoretică

1.1 Ce este programarea vizuală?

Programarea vizuală permite construirea interfeței unei aplicații prin elemente grafice: ferestre, butoane, liste și panouri. În Visual Studio, pot așeza un control pe formular și îi pot modifica dimensiunea sau culoarea fără să scriu manual fiecare instrucțiune de configurare. Designer-ul salvează aceste alegeri în cod C#. [1, 2]

Configurarea vizuală stabilește *cum arată* interfața. Codul scris de programator stabilește *ce se întâmplă* când utilizatorul interacționează cu ea. De exemplu, butonul PLAY poate fi desenat și poziționat în Designer, dar pornirea unei runde este realizată de metoda C# asociată evenimentului Click.

Acest mod de lucru se folosește la aplicații de evidență, formulare de înscriere, calculatoare, editoare și proiecte educaționale. În Zombie Survivors, aceeași abordare este aplicată meniurilor și indicatoarelor jocului.

1.2 Ce este Windows Forms?

Windows Forms este o tehnologie .NET pentru aplicații desktop pe Windows. O fereastră este reprezentată printr-un obiect Form, iar în interiorul ei pot exista controale precum Button, Label, Panel sau ComboBox. Controalele expun proprietăți, metode și evenimente. Pentru desene proprii, formularul oferă acces la obiectul Graphics. [1]

Pentru proiectul meu, avantajul principal este că pot vedea imediat structura interfeței și pot conecta fiecare buton la o metodă. Folosesc controale standard pentru meniuri și HUD, iar arena și personajele sunt desenate separat. Astfel, pot exersa atât programarea interfeței, cât și organizarea logicii în clase.

1.3 Limbajul C# și platforma .NET

C# este limbajul în care este scris proiectul, iar .NET oferă mediul de execuție și bibliotecile folosite. Fișierul proiectului declară ținta **net10.0-windows** și opțiunea **UseWindowsForms**. Aceasta este configurația reală analizată, nu o aplicație .NET Framework mai veche. [3, 4]

Noțiune	**Exemplu concret în proiect**
Clasă și obiect	Player descrie un jucător; session.Player este obiectul folosit în rundă.
Variabilă / proprietate	Health păstrează viața curentă, iar Level păstrează nivelul.
Metodă	Reset() reinițializează sesiunea; UpdateHealth() actualizează HUD-ul.
Eveniment	Click anunță apăsarea unui buton; Tick declanșează actualizarea periodică.

Cerințele aplicației

1.4 Cerințe funcționale

O cerință funcțională descrie o acțiune pe care aplicația trebuie să o realizeze. În acest proiect, jucătorul se deplasează cu WASD, ține cu mouse-ul și trage cu butonul stâng. Inamicii apar în valuri, urmăresc jucătorul și oferă XP atunci când sunt eliminați. La creșterea nivelului se afișează trei opțiuni de upgrade.

Interfața trebuie să permită pornirea jocului, pauza, alegerea modului de afișare și reluarea unei runde după Game Over. Bara HP, progresul XP, nivelul și numărul inamicilor trebuie să reflecte datele curente ale sesiunii.

1.5 Cerințe non-funcționale

O cerință non-funcțională descrie calitatea utilizării: claritate, stabilitate, performanță sau compatibilitate. Pentru un joc cu multe actualizări, interfața trebuie să rămână lizibilă, iar obiectele temporare trebuie eliminate când nu mai sunt necesare. Compatibilitatea vizată este Windows, conform configurației proiectului.

Timer-ul are intervalul configurat la 16 ms, adică un ritm nominal de aproximativ 62,5 actualizări pe secundă. Această setare nu demonstrează un număr garantat de cadre: încărcarea calculatorului și mesajele interfeței pot întârzia Tick. În lucrare descriu mecanismele existente, fără a atribui rezultate de performanță nemăsurate.

1.6 Tabel comparativ

Tip	Cerință	Realizare în proiect
Funcțională	Deplasare și țintire	GameInputController citește tastele; Player actualizează poziția și unghiul.
Funcțională	Luptă și inamici	CombatSystem gestionează proiectilele; EnemyManager actualizează inamicii.
Funcțională	Progresie	XP, niveluri și o alegere de upgrade pentru fiecare nivel câștigat.
Funcțională	Navigare în meniuri	ScreenController afișează panoul corespunzător stării curente.
Non-funcțională	Claritatea interfeței	Text alb pe panouri închise; bare distincte: roșu pentru HP și cyan pentru XP.
Non-funcțională	Stabilitate la resetare	O rundă nouă golește listele de obiecte și resetează progresul.
Non-funcțională	Utilizarea resurselor	Resursele bitmap sunt reutilizate; renderer-ul eliberează obiectele proprii.

Tip	Cerință	Realizare în proiect
Non-funcțională	Adaptarea ferestrei	Anchor, Dock și recentrarea panourilor gestionează redimensionarea.

2. Prezentarea aplicației

2.1 Descrierea jocului

Zombie Survivors este un joc 2D de supraviețuire realizat în C# Windows Forms. Scopul unei runde este să rezist cât mai mult, să elimin inamicii și să îmbunătățesc personajul. Dificultatea crește prin numărul inamicilor din valuri și prin introducerea unor tipuri mai rezistente.

2.2 Funcționalități principale

WASD controlează deplasarea, **mouse-ul** stabilește ținta, iar **clicul stâng** încearcă să tragă un proiectil. **ESC** deschide sau închide pauza, iar **F3** afișează Debug Menu. Jocul include HP, XP, Level Up, upgrade-uri, bandaje, Settings, Game Over și Respawn.

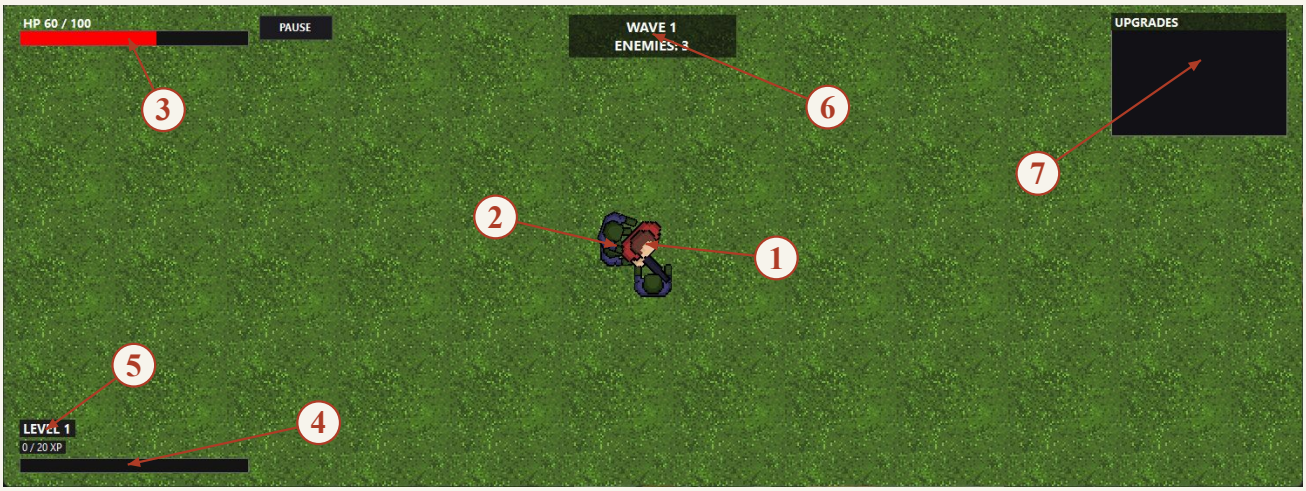
2.3 Tehnologii utilizate

Visual Studio oferă editorul de cod, Designer-ul și instrumentele de proiect. C# exprimă regulile jocului, .NET și Windows Forms furnizează fereastra și controalele, iar Git și GitHub păstrează istoricul și repository-ul proiectului.

2.4 Captură generală a jocului

Săgețile indică: 1 - jucătorul; 2 - inamicii; 3 - bara HP; 4 - bara XP; 5 - nivelul; 6 - valul și inamicii rămași; 7 - lista de upgrade-uri.

Interfața principală a jocului Zombie Survivors; captură reală dintr-o rundă.



Soluția, namespace-urile și titlul ferestrei mai conțin denumiri vechi. Capturile și fragmentele sunt autentice, de aceea le păstrez. Numele utilizat în această lucrare este Zombie Survivors.

3. Structura proiectului

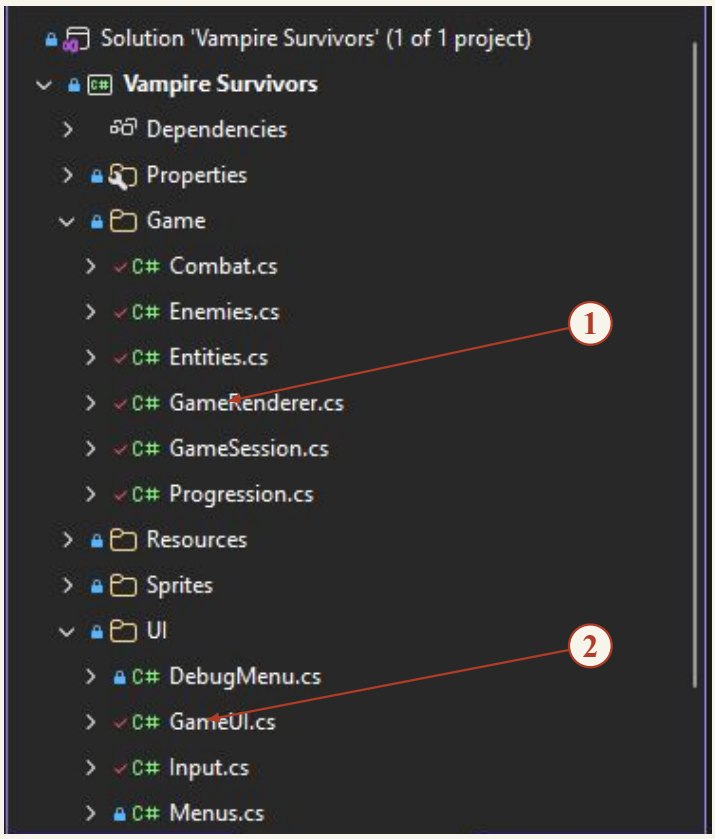
3.1 Organizarea fișierelor

Game/ conține Entities.cs (Player, statistici, proiectile și efecte), Enemies.cs (inamici și valuri), Combat.cs, GameSession.cs, GameRenderer.cs și Progression.cs.

UI/ conține GameUI.cs (HUD și Level Up), Menus.cs (ecrane și afișare), Input.cs (tastatură și mouse) și DebugMenu.cs.

Form1.cs conectează aceste clase. **Form1.Designer.cs** definește controalele. **Program.cs** pornește aplicația. **Properties/Resources.resx** și directoarele cu imagini furnizează resursele grafice.

Fișierele actuale din Game și UI, în Solution Explorer.



3.2 Rolul formularului Form1

Form1 este fereastra principală și punctul de întâlnire al componentelor. În constructor creez sesiunea, renderer-ul și controlerele pentru HUD, meniuri, intrare și Debug. Formularul primește evenimentele Windows Forms și le transmite obiectului responsabil, fără să păstreze toate regulile de luptă în același fișier.

Clasa este **partial**: Form1.cs și Form1.Designer.cs reprezintă două părți ale aceleiași clase. Această separare permite Designer-ului să gestioneze configurația controalelor, iar codului scris manual să organizeze comportamentul lor.

3.3 Legătura dintre componente

Interfață / intrare → Eveniment C# → GameSession → HUD + randare

Un Click sau o tastă produce un eveniment. Controlerul verifică starea ecranului și apelează logica sesiunii. Ulterior, HUD-ul citește valorile noi, iar Invalidate() solicită redesenarea prin OnPaint. În timpul jocului, Timer-ul repetă ciclul de actualizare.

4. Realizarea practică a interfeței

4.1 Crearea proiectului Windows Forms

Pentru a realiza un proiect de acest tip în Visual Studio, deschid **Create a new project**, aleg șablonul **Windows Forms App** pentru limbajul C#, stabilesc numele și folderul soluției, apoi aleg versiunea .NET. Proiectul analizat folosește .NET 10 pentru Windows. Pașii descriu modul de creare; capturile din lucrare prezintă proiectul existent. [4]

După creare, deschid Form1 în modul **Designer**. Suprafața formularului îmi permite să așez controale din Toolbox. Pentru configurarea unui control folosesc fereastra Properties. Pentru comportament trec în Form1.cs și scriu metodele asociate evenimentelor.

Fișier	**Rol la pornire și în interfață**
Program.cs	Main() inițializează configurația aplicației și execută Application.Run(new Form1()).
Form1.Designer.cs	Declară câmpurile controalelor și implementează InitializeComponent(), cu obiecte, proprietăți, ierarhie și evenimente.
Form1.cs	Apelează InitializeComponent(), construiește controlerele și conectează logica jocului la formular.

Codul 1. Inițializarea formularului

Form1.cs:20-24
Form1() • începutul constructorului; continuarea este omisă

```
public Form1()
{
    InitializeComponent();

    DoubleBuffered = true;
```

InitializeComponent() trebuie executată înainte ca programul să folosească lblHealth, btnPlay sau celelalte controale. Ea creează obiectele și le adaugă în containerele potrivite. Dacă un eveniment din Designer indică o metodă care nu mai există, proiectul sau Designer-ul pot raporta o eroare.

DoubleBuffered activează desenarea intermediară într-un buffer. În acest joc ajută la reducerea pâlpâirii în timpul redesenării arenei. Proprietatea nu înlocuiește actualizarea logicii și nu fixează numărul de cadre pe secundă.

De la formular gol la interfață

Am organizat interfața în panouri: meniul principal, setările, pauza, Level Up și Game Over. Pe formular rămân și controalele HUD-ului. Panourile reprezintă ecrane ale aceleiași ferestre; navigarea schimbă vizibilitatea lor, fără să deschidă un formular nou pentru fiecare meniu.

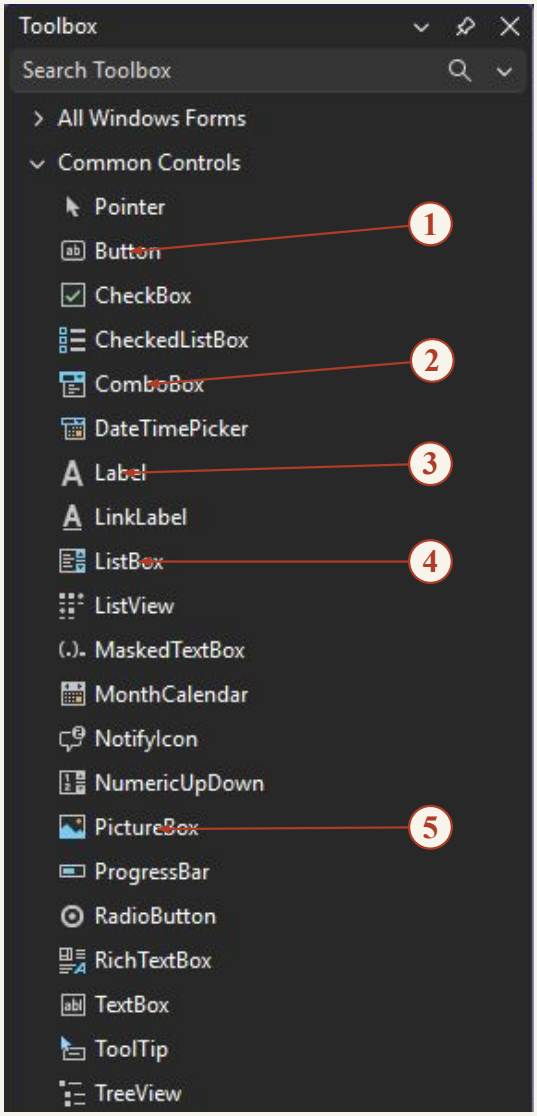
Mai întâi stabilesc containerele Panel, apoi adaug textele și butoanele, configurez proprietățile și, la final, conectez evenimentele. Ierarhia părinților este importantă: Location este relativă la containerul controlului.

4.2 Utilizarea Toolbox-ului

Toolbox este lista de controale și componente disponibile pentru Designer. Îl pot deschide din View → Toolbox sau prin Ctrl+Alt+X. Aleg un element și îl plasez pe formular ori în interiorul unui Panel. Controlul selectat se configurează apoi în Properties. [2, 4]

Control	**Utilizare efectivă**
Button	PLAY, SETTINGS, RESUME, RESPAWN și cele trei upgrade-uri.
Label	HP, XP, nivel, val, titluri și informații Debug.
Panel	Ecrane, grupuri de controale și cele două straturi ale barelor.
PictureBox	Logo-ul meniului: Image din resurse, SizeMode = Zoom.
CheckBox	Infinite Health și Infinite XP în Debug.
ComboBox	Lista închisă a modurilor de afișare.
ListBox	Upgrade-urile deținute, în partea dreaptă-sus.
Timer	Declanșează periodic evenimentul Tick al jocului.

Toolbox deschis în Visual Studio: 1 - Button; 2 - ComboBox; 3 - Label; 4 - ListBox; 5 - PictureBox.



De ce am ales aceste controale

Button oferă direct evenimentul Click. Label este potrivit pentru informații scurte, iar ListBox poate afișa un număr variabil de upgrade-uri. Panel grupează controalele și permite ascunderea întregului meniu. ComboBox cu DropDownList împiedică introducerea unei valori care nu există în listă.

Pentru fiecare control stabilesc un **Name** clar, apoi Text, Size, Location, culori și font. La butoane configurez și aspectul la trecerea mouse-ului. Pentru controalele HUD aleg Anchor în funcție de marginea ferestrei lângă care trebuie să rămână.

Este o componentă Windows Forms, dar gameTimer este construit în Form1.cs, cu Interval = 16 și Tick += GameTimer_Tick. Nu este un control vizibil și nu a fost adăugat în fișierul Designer.

4.3 Form Designer și Properties

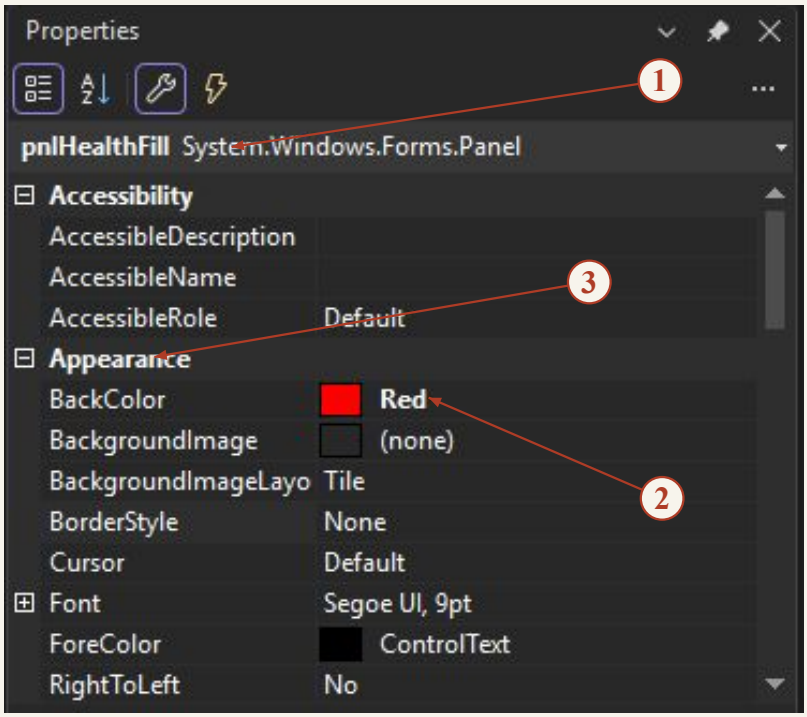
În Designer selectez un control pentru a-i vedea conturul și proprietățile. Properties modifică configurația obiectului selectat; Document Outline arată ierarhia, utilă când mai multe panouri se suprapun. În captura de mai jos este selectat pnlHealthFill, de tip Panel.

Name este numele folosit în cod, de exemplu pnlHealthFill. **Text** este conținutul afișat, ca PLAY sau HP 100 / 100. Un Panel folosit ca bară nu are nevoie de text.

Size înseamnă lățime și înălțime. **Location** este poziția în interiorul părintelui. pnlHealthFill începe la (0, 0) în panoul de fundal; nu folosește coordonatele întregului ecran.

BackColor colorează fundalul, **ForeColor** textul, iar **Font** stabilește familia, mărimea și stilul literelor.

Properties pentru pnlHealthFill: 1 - obiectul selectat; 2 - BackColor = Red; 3 - categoria Appearance.



Proprietate	**Exemplu și efect în Zombie Survivors**
Anchor	Bottom | Left pentru lblLevel, lblXp și pnlXpBackground; păstrează distanța față de stânga și de jos.
Dock	Fill pentru pnlMainMenu și pnlSettings; panoul ocupă suprafața disponibilă a formularului.
Visible	pnlLevelUp pornește ascuns; ScreenController îl afișează la alegerea unui upgrade.
Font / culori	btnPlay folosește Segoe UI 12, Bold, text alb și fundal închis.
Size / Location	picLogo are 350 × 150 și poziția (44, 22) în pnlMenuContent, în configurația Designer.

Click este un eveniment

Click nu este o proprietate obișnuită, precum BackColor. Este un eveniment. În Properties trec la lista de evenimente, prin pictograma fulger, și asociez o metodă. Legătura reală a butonului PLAY este **btnPlay.Click += BtnPlay_Click;**. La apăsare, Windows Forms execută metoda respectivă. [5]

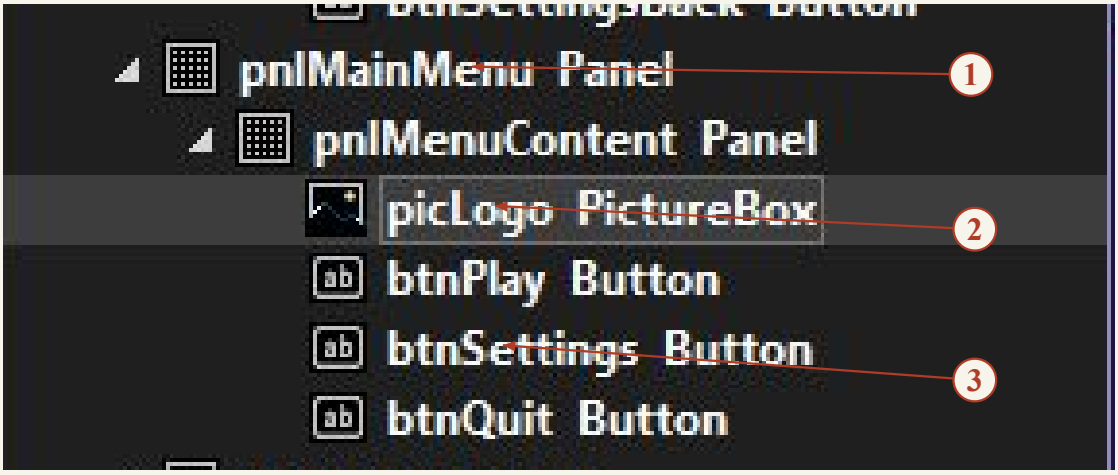
Controalele ascunse în timpul rulării pot fi totuși vizibile sau suprapuse în Designer. Aspectul de proiectare nu trebuie confundat cu starea curentă a jocului.

4.4 Crearea meniului principal

Am grupat meniul în **pnlMainMenu**, cu Dock = Fill. În interior se află **pnlMenuContent**, care conține picLogo, btnPlay, btnSettings și btnQuit. ScreenController recentrează grupul după dimensiunea ferestrei; pozițiile butoanelor rămân relative la acest grup.

În Designer, logo-ul selectat aparține meniului chiar dacă alt panou îl acoperă pe suprafața de proiectare.

Meniul în Document Outline al Designer-ului: 1 - container; 2 - PictureBox; 3 - butoane.



Meniul la rulare: logo, PLAY, SETTINGS și QUIT.



picLogo încarcă Properties.Resources.Logo. Modul Zoom păstrează proporțiile imaginii în spațiul disponibil. Logo-ul este static în versiunea actuală; nu există o animație de meniu care trebuie descrisă.

Codul 2. Acțiunea butonului PLAY

Form1.cs:104-104; Form1.cs:147-151
BtnPlay_Click() și StartNewGame()

```
private void BtnPlay_Click(object? sender, EventArgs e) => StartNewGame();
// ... porțiune intermediară omisă ...
private void StartNewGame()
{
    RestartGame();
    gameTimer.Start();
}
```

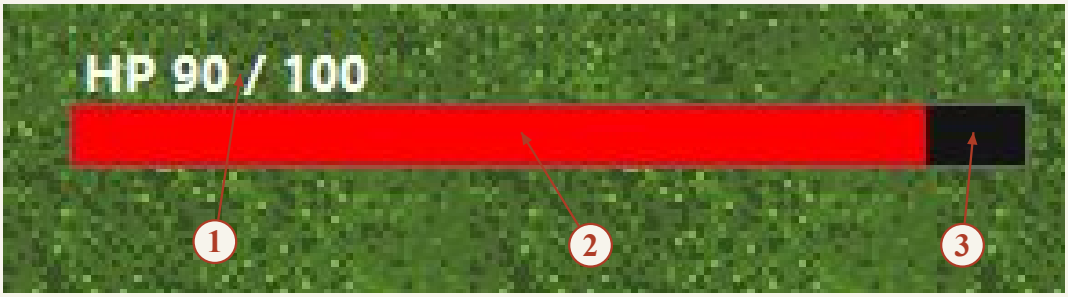
Evenimentul Click ajunge în **BtnPlay_Click**, care apelează StartNewGame(). Mai întâi RestartGame() pregătește sesiunea, apoi gameTimer.Start() pornește actualizările. SETTINGS afișează panoul de setări, iar QUIT execută Application.Exit().

Butoanele folosesc FlatStyle = Flat, Cursor = Hand, text alb și culori distincte la hover sau apăsare. Aceste proprietăți oferă feedback fără să necesite desenarea manuală a fiecărui buton.

4.5 Crearea barei HP

Pentru bara de viață am folosit două controale Panel. **pnlHealthBackground** este fundalul întunecat, cu bordură, iar **pnlHealthFill** este partea roșie din interior. Deasupra se află **lblHealth**, care afișează viața curentă și limita maximă.

Bara HP după o lovitură: 1 - textul; 2 - umplerea roșie; 3 - fundalul rămas liber.



În Designer, fundalul are Size = 263 × 18, iar umplerea începe la Location = (0, 0). Culoarea umplerii este Red. Lățimea inițială din Designer este numai o valoare de proiectare: la rulare, metoda UpdateBar() o calculează din viața jucătorului.

Codul 3. Actualizarea vieții și a lățimii barei

UI/GameUI.cs:41-48; UI/GameUI.cs:65-71
HudController.UpdateHealth() și UpdateBar()

```
public bool UpdateHealth()
{
    Player player = session.Player;
    UpdateBar(healthFill, player.Health, player.MaxHealth);
    healthLabel.Text = $"HP {player.Health} / {player.MaxHealth}";
    return player.IsDead;
}

// ... porțiune intermediară omisă ...
private static void UpdateBar(Panel fill, int value, int maximum)
{
    // Resize only the fill panel; the labels stay above the bar.
    float fraction = maximum > 0 ? value / (float)maximum : 0f;
    fraction = Math.Clamp(fraction, 0f, 1f);
    fill.Width = (int)(fill.Parent!.ClientSize.Width * fraction);
}
```

player.Health reprezintă HP curent, iar **player.MaxHealth** limita dată de statistici. Formula este **procent HP = HP curent / HP maxim**. Conversia la float permite un rezultat fracționar. Math.Clamp limitează rezultatul între 0 și 1, iar verificarea maximum > 0 evită împărțirea la zero.

Procentul înmulțește lățimea interioară a panoului părinte. De exemplu, pentru 90 / 100 HP și o zonă interioară de 261 px, lățimea calculată este 234 px după conversia la int. Se schimbă numai umplerea; label-ul și fundalul își păstrează poziția.

UpdateHealth() returnează și player.IsDead. GameTimer_Tick folosește rezultatul primit prin RefreshFrame() pentru a afișa panoul Game Over când viața ajunge la zero.

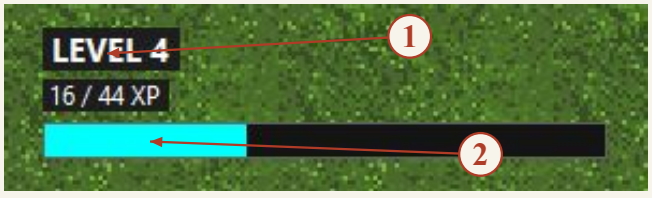
4.6 Bara XP și nivelul

În partea stângă-jos am plasat **lblLevel**, **lblXp** și **pnlXpBackground**. În panoul de fundal se află pnlXpFill, de culoare Cyan. Cele trei controale exterioare au Anchor = Bottom | Left; astfel își păstrează distanțele față de marginile de jos și din stânga la redimensionare.

La pornire: nivel 1; săgeata 1 indică 0 / 20 XP.



După +100 XP: 1 - nivelul 4; 2 - bara 16 / 44 XP.



HudController.UpdateExperience() citește Player, actualizează textele și apelează aceeași metodă UpdateBar() ca la HP. Aici fracția este **Experience / ExperienceToNextLevel**. După eliminarea unui inamic, CombatSystem acordă XP, iar sesiunea înregistrează nivelurile nou câștigate.

Codul 4. Calculul progresului în Player

Game/Entities.cs:158-176
Player.AddExperience()

```
public int AddExperience(int amount)
{
    if (amount <= 0 || IsDead)
        return 0;

    Experience += amount;
    int levelsGained = 0;

    // Keep leftover XP, even if one reward gains several levels.
    while (Experience >= ExperienceToNextLevel)
    {
        Experience -= ExperienceToNextLevel;
        Level++;
        ExperienceToNextLevel += 8;
        levelsGained++;
    }

    return levelsGained;
}
```

amount este recompensa primită. **Experience** păstrează XP acumulat, iar **ExperienceToNextLevel** începe de la 20 și crește cu 8 după fiecare nivel. Bucla while păstrează surplusul: o recompensă mare poate acorda mai multe niveluri, fără să piardă XP rămas.

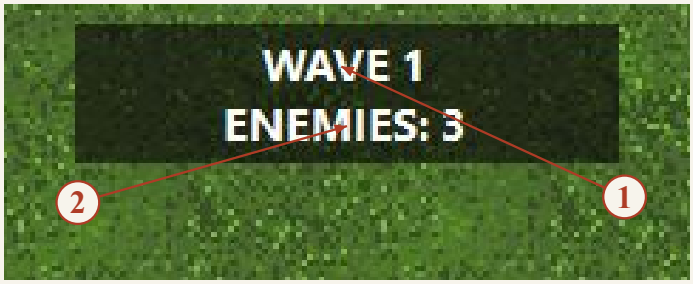
În captura reală, comanda Debug +100 XP a trecut prin pragurile 20, 28 și 36: 100 − 20 − 28 − 36 = 16. De aceea HUD-ul arată LEVEL 4 și 16 / 44 XP. Meniurile de alegere pentru nivelurile 2, 3 și 4 sunt prezentate pe rând, prin coada PendingLevels.

Player.Reset() readuce Level la 1, Experience la 0 și pragul la 20. RestartGame() cere imediat actualizarea HUD-ului, pentru ca valorile vechi să nu rămână pe ecran.

4.7 Interfața pentru valuri

Pentru valuri folosesc un singur **Label**, lblWave, cu două rânduri de text și aliniere centrală. Primul rând indică numărul valului, iar al doilea numărul inamicilor rămași. În pauza dintre valuri, același label anunță finalizarea și timpul până la următorul val.

Detaliu al lblWave: 1 - valul curent; 2 - totalul inamicilor rămași în val.



Codul 5. Actualizarea informațiilor despre val

UI/GameUI.cs:73-81
HudController.UpdateWave()

```
public void UpdateWave()
{
    WaveManager waves = session.Waves;
    int seconds = (waves.IntermissionRemainingMs + 999) / 1000;
    waveLabel.Text = waves.IsWaveActive
        ? $"WAVE {waves.CurrentWave}{Environment.NewLine}ENEMIES:
    {waves.GetEnemiesRemaining(session.Enemies.Count)}"
        : $"WAVE {waves.CurrentWave} CLEARED{Environment.NewLine}NEXT WAVE:
    {seconds}s";
}
```

session.Waves este managerul valurilor. **CurrentWave** este numărul valului, iar **IsWaveActive** selectează textul afișat. Environment.NewLine separă rândurile. Pentru pauză, expresia (milisecunde + 999) / 1000 rotunjește în sus numărul de secunde, astfel încât un rest de timp să nu fie afișat ca zero.

Element	Semnificație pentru interfață
WAVE 1	Valul din care fac parte inamicii și aparițiile încă programate.
ENEMIES: 3	Include inamicii vii și pe cei din val care încă nu au apărut.
WAVE … CLEARED	Valul s-a încheiat și se execută pauza dintre valuri.
NEXT WAVE: …s	Timpul rămas până la începerea următorului val.

Poziționarea la redimensionare

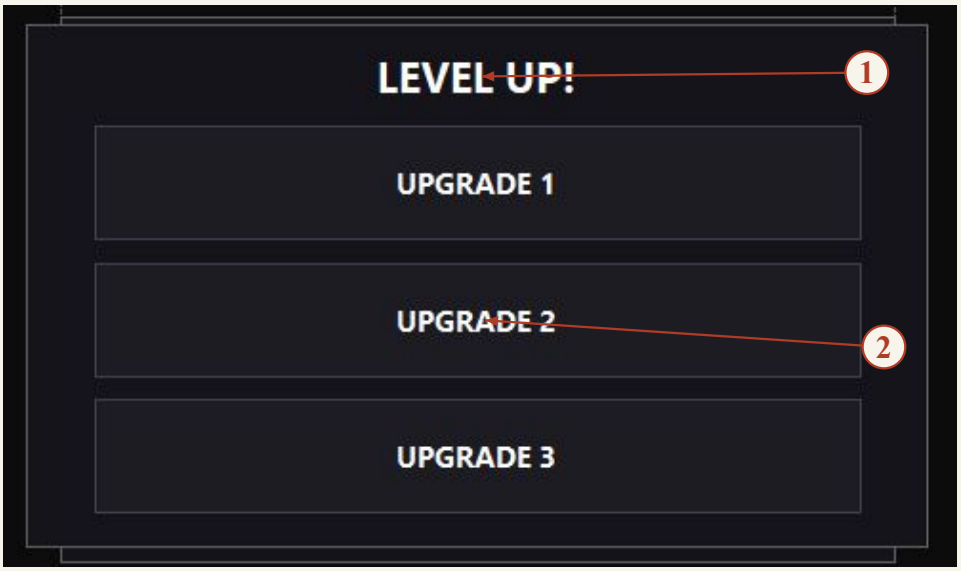
HudController.Layout() centrează label-ul pe orizontală. Dacă spațiul este prea mic lângă butonul Pause sau lista de upgrade-uri, mută textul mai jos. Astfel, poziția este stabilită după dimensiunea curentă a ferestrei, nu doar după coordonatele inițiale din Designer.

Label-ul nu creează inamici și nu avansează valul. El citește datele din WaveManager. Această separare face mai ușoară modificarea textului fără schimbarea regulilor jocului.

4.8 Interfața Level Up

Panoul **pnlLevelUp** conține lblLevelUpTitle și trei Button: btnUpgrade1, btnUpgrade2, btnUpgrade3. Are fundal închis, bordură și Visible = false în configurația inițială. ScreenController îl centrează și îl aduce în față când starea devine LevelUp.

Panoul Level Up în Designer: titlu și trei butoane.



Opțiuni generate la rulare pentru nivelul 2.



Codul 6. Completarea textelor celor trei butoane

UI/GameUI.cs:150-162
LevelUpController.ShowNext() • fragmentul care pregătește opțiunile

```
int level = session.PendingLevels.Peek();
// Every level offers three different normal upgrades.
upgradeChoices = session.Upgrades.GetRandomUpgrades(buttons.Length);
title.Text = $"LEVEL {level} — CHOOSE AN UPGRADE";
for (int index = 0; index < buttons.Length; index++)
{
    Button button = buttons[index];
    button.Visible = true;
    UpgradeType choice = upgradeChoices[index];
    (string name, string description) = UpgradeSystem.GetUpgradeInfo(choice);
    button.Text = $"{name}{Environment.NewLine}{description}   Owned:
    {session.Upgrades.GetStackCount(choice)}";
}
screens.ShowLevelUp();
```

PendingLevels.Peek() indică nivelul pentru care trebuie făcută alegerea. **buttons.Length** este 3. GetRandomUpgrades() alege din lista upgrade-urilor normale și scoate opțiunea aleasă din grupul temporar, astfel încât același meniu să nu conțină duplicate.

În buclă, fiecare buton devine vizibil și primește numele upgrade-ului, descrierea și numărul de exemplare deținute, afișat prin **Owned**. Textele inițiale UPGRADE 1-3 sunt doar valori de proiectare; la rulare sunt înlocuite cu informațiile opțiunilor reale.

Cât timp starea este LevelUp, GameTimer_Tick se oprește înaintea actualizării sesiunii. Jucătorul poate citi și alege fără ca inamicii să continue să-l atace.

Aplicarea și afișarea upgrade-urilor

Fiecare buton are propriul eveniment Click. Metodele BtnUpgrade1_Click, BtnUpgrade2_Click și BtnUpgrade3_Click transmit indicii 0, 1 și 2 către ChooseLevelUpOption(). Alegerea ajunge apoi în LevelUpController.Choose(), care validează starea înainte să modifice jucătorul.

Codul 7. Aplicarea alegerii și continuarea meniurilor

UI/GameUI.cs:165-179
LevelUpController.Choose()

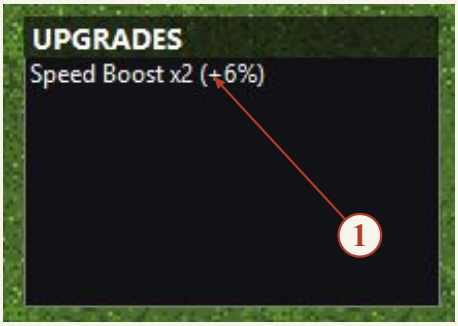
```
public bool Choose(int index, bool infiniteXp)
{
    if (screens.State != GameState.LevelUp || session.PendingLevels.Count == 0 ||
        index < 0 || index >= upgradeChoices.Count)
        return false;
    session.Upgrades.ApplyUpgrade(upgradeChoices[index], session.Player);
    session.PendingLevels.Dequeue();
    if (infiniteXp)
        session.GrantNextLevel();
    hud.UpdateHealth();
    hud.UpdateExperience();
    hud.UpdateUpgrades();
    ShowNext();
    return true;
}
```

index este poziția butonului ales, iar **upgradeChoices** conține opțiunile curente. ApplyUpgrade() modifică statisticile și numărul de stive; Dequeue() elimină nivelul rezolvat. La final sunt actualizate HP, XP și lista de bonusuri, apoi ShowNext() verifică dacă mai există o alegere în așteptare.

lstUpgrades este un ListBox, ancorat sus-dreapta. Fiecare tip de upgrade apare pe un singur rând, cu numărul de stive și bonusul total.

HudController.UpdateUpgrades() citește Version. Dacă versiunea nu s-a schimbat, lista nu este reconstruită inutil la fiecare Tick.

Lista din dreapta-sus după două alegeri Speed Boost: x2, total +6%.



Când dispare panoul?

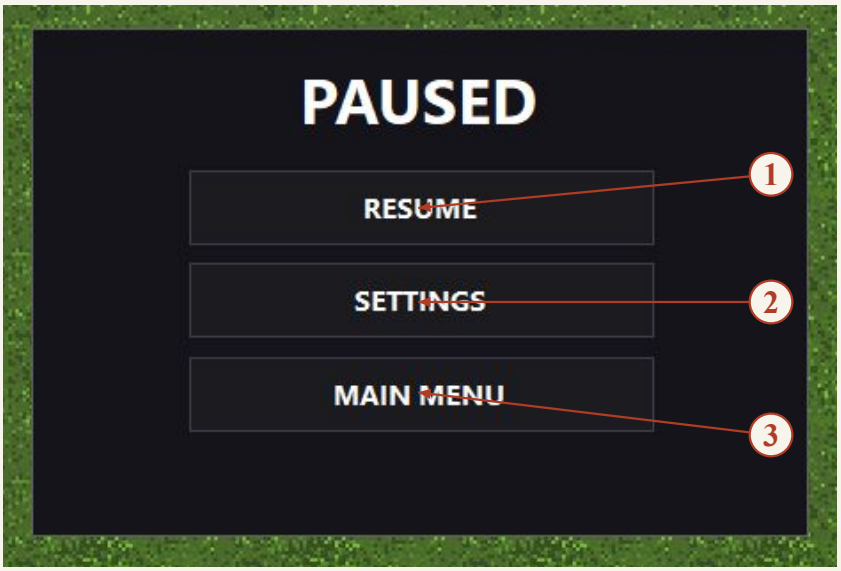
Dacă mai există niveluri în coadă, ShowNext() pregătește un nou set de trei opțiuni. Dacă nu mai există, CloseLevelUp() trece la Playing și ascunde panoul. Când jucătorul este mort, aceeași verificare duce la GameOver. În modul Debug, infiniteXp poate acorda încă un nivel după fiecare alegere.

Speed Boost adaugă de fiecare dată 3% la bonusul de viteză. Două alegeri dau +6%, nu 1,03 × 1,03. La viteza de bază 6, rezultatul este 6 × 1,06 = 6,36 unități pe actualizare.

4.9 Pause Menu

Meniul de pauză este **pnlPause**, cu lblPaused și butoanele RESUME, SETTINGS și MAIN MENU. Este centrat de ScreenController. Același meniu se poate deschide din butonul PAUSE al HUD-ului sau prin ESC în timpul jocului.

Meniul Pause: 1 - RESUME; 2 - SETTINGS; 3 - MAIN MENU.



Codul 8. Schimbarea stării pentru pauză și reluare

UI/Menus.cs:63-78
ScreenController.Pause() și Resume()

```
public bool Pause()
{
    if (State != GameState.Playing)
        return false;
    SetState(GameState.Paused);
    return true;
}

public bool Resume()
{
    if (State != GameState.Paused)
        return false;
    SetState(GameState.Playing);
    return true;
}
```

Pause() acceptă cererea doar din Playing, iar Resume() doar din Paused. În Form1, PauseGame() și ResumeGame() șterg tastele de mișcare reținute. Astfel, reluarea nu pornește o deplasare rămasă dinaintea pauzei. La reluare se verifică și existența nivelurilor în așteptare.

Pauza nu înseamnă oprirea și recrearea Timer-ului. În GameTimer_Tick, prima verificare returnează imediat dacă starea nu este Playing. Datele rundei rămân în memorie, dar mișcarea, lupta

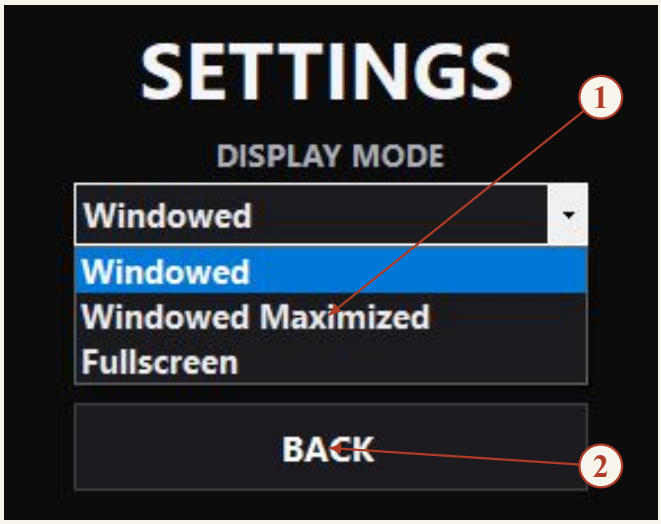
și valurile nu mai sunt actualizate. KeyDown gestionează ESC o singură dată pentru fiecare apăsare, folosind setul de taste apăsate.

SETTINGS reține că revenirea trebuie făcută la pauză. MAIN MENU oprește Timer-ul, reinițializează sesiunea și afișează meniul principal. RESUME continuă runda curentă.

4.10 Settings

Setările sunt grupate în **pnlSettings** și pnlSettingsContent. Panoul interior conține titlul, eticheta DISPLAY MODE, **cmbDisplayMode** și butoanele APPLY și BACK. ComboBox are DropDownStyle = DropDownList, deci utilizatorul poate alege numai una dintre cele trei valori existente.

Setările reale: 1 - lista celor trei moduri; 2 - butonul BACK.



Mod	Efect implementat de DisplayManager
Windowed	Formular cu bordură redimensionabilă; poziția și dimensiunea ferestrei sunt memorate pe durata sesiunii aplicației.
Windowed Maximized	Formular cu bordură redimensionabilă și WindowState = Maximized.
Fullscreen	Formular fără bordură, așezat pe dreptunghiul complet al monitorului curent.

Aplicarea unei alegeri

Click pe APPLY ajunge în BtnApplySettings_Click(), apoi în display.ApplySelectedMode(). SelectedIndex este convertit la DisplayMode, iar Apply() modifică FormBorderStyle, WindowState și Bounds. Pentru Windowed, dimensiunea și poziția sunt limitate la zona de lucru disponibilă a monitorului.

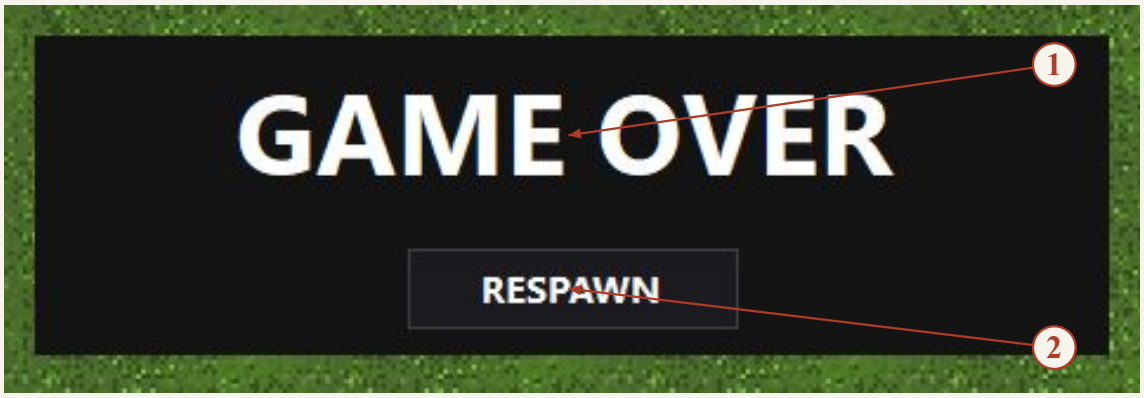
Variabila **applying** împiedică tratarea schimbărilor intermediare de dimensiune ca modificări făcute manual. La redimensionare, Form1 cere recentrarea panourilor și actualizarea dimensiunii arenei. Nu există în acest ecran setări de volum sau de dificultate.

Din meniul principal, BACK revine la MainMenu. Dacă Settings a fost deschis din Pause, BACK revine la Paused. Această alegere este păstrată în SettingsReturnToPause; nu se pierde runda aflată în pauză.

4.11 Game Over și Respawn

Când HP ajunge la zero, Player.IsDead devine true. Actualizarea HUD-ului raportează moartea, iar Form1 apelează screens.ShowGameOver(). **pnlGameOver** afișează titlul și butonul RESPAWN. Starea GameOver oprește actualizările de joc prin aceeași verificare de la începutul Tick-ului.

Game Over real: 1 - mesajul; 2 - butonul RESPAWN.



Codul 9. Reinițializarea sesiunii și a HUD-ului

Form1.cs:162-176
Form1.RestartGame()

```
private void RestartGame()
{
    // The form may finish maximizing after the session was created.
    session.Resize(ClientSize);
    session.Reset();
    gameInput.Reset();
    screens.SetPlaying();
    hud.UpdateHealth();
    hud.UpdateExperience();
    hud.UpdateUpgrades();
    hud.UpdateWave();
    debug.UpdateInfo();
    Invalidate();
}
```

session.Resize(ClientSize) folosește dimensiunea actuală a ferestrei. Reset() pornește o sesiune curată, iar gameInput.Reset() elimină tastele reținute. SetPlaying() ascunde Game Over și readuce starea de joc. Cele patru apeluri HUD afișează imediat viața, XP, upgrade-urile și valul resetat.

Date resetate	Rezultatul unei runde noi
Jucător și progres	HP 100 / 100, nivel 1, 0 / 20 XP, statistici de bază și fără upgrade-uri.

Date resetate	Rezultatul unei runde noi
Lumea rundei	Fără inamici vechi, bandaje, proiectile, texte de luptă sau particule.
Valuri și alegeri	Valul 1, contorul de apariții resetat, PendingLevels golit.

BtnRespawn_Click() apelează RestartGame(). Timer-ul rămâne pornit în GameOver, iar revenirea la Playing permite actualizarea. Fluxul complet este explicat în subcapitolul 4.13.

4.12 Debug Menu

Meniul Debug se deschide prin **F3** și este un Panel cu etichete, două CheckBox-uri și cinci butoane. DebugController.Toggle() îl ascunde sau îl afișează și îl aduce în față. Nu îl deschide din MainMenu sau Settings.

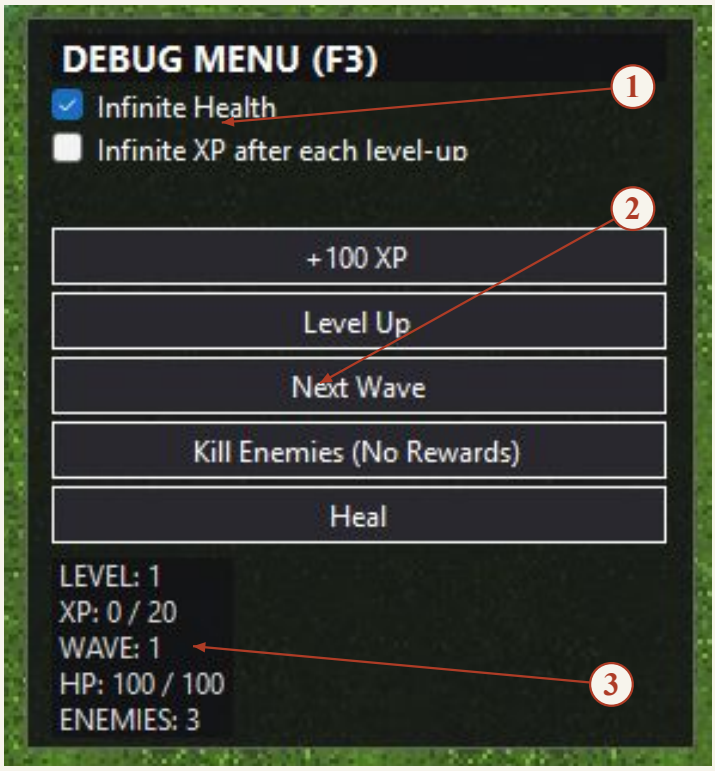
Infinite Health previne damage-ul de contact și reface viața dacă personajul este încă viu.

Infinite XP acordă încă un nivel după fiecare alegere de upgrade. Nu este o valoare numerică infinită în proprietatea XP.

+100 XP / Level Up acordă experiență sau exact cât lipsește pentru pragul următor.

Next Wave curăță inamicii și avansează valul. **Kill Enemies (No Rewards)** încheie valul fără recompense. **Heal** vindecă un personaj viu.

Meniul Debug real: 1 - CheckBox-uri; 2 - instrumente; 3 - datele sesiunii.



Codul 10. Acordarea de XP prin interfața Debug

UI/DebugMenu.cs:55-65
DebugController.AddExperience()

```
public void AddExperience(int amount)
{
    if (amount <= 0 || session.Player.IsDead)
        return;
    session.AddExperience(amount);
    hud.UpdateExperience();
    if (session.PendingLevels.Count > 0 && screens.State == GameState.Playing)
        levels.ShowNext();
    UpdateInfo();
    repaint();
}
```

Metoda ignoră valorile nepozitive și jucătorul mort. După session.AddExperience(amount), actualizează bara XP și deschide Level Up dacă există alegeri și jocul este Playing. Apoi reîmprospătează informațiile Debug și solicită redesenarea prin callback-ul repaint().

Acest meniu permite verificarea rapidă a nivelurilor, stivelor și valurilor. Capturile progresiei din lucrare au fost obținute prin aceste controale existente, fără schimbarea surselor sau simularea unor ecrane.

4.13 Legătura dintre UI și cod

În Windows Forms, un buton nu funcționează doar pentru că apare pe formular. El trebuie să existe ca obiect, să fie adăugat într-un container și să aibă evenimentul conectat. Pentru PLAY pot urmări toate aceste etape în fișierele reale ale proiectului.

Declarare, creare și proprietăți

În Form1.Designer.cs, câmpul este declarat **private System.Windows.Forms.Button btnPlay;**. În InitializeComponent() este creat prin **btnPlay = new Button();** și adăugat în pnlMenuContent. Aceste instrucțiuni aparțin aceleiași clase partial Form1 folosite și de codul manual.

Codul 11. Proprietățile butonului și legarea evenimentului

Form1.Designer.cs:302-317
InitializeComponent() • configurația reală a btnPlay

```
btnPlay.BackColor = Color.FromArgb(28, 28, 32);
btnPlay.Cursor = Cursors.Hand;
btnPlay.FlatAppearance.BorderColor = Color.FromArgb(65, 65, 75);
btnPlay.FlatAppearance.MouseDownBackColor = Color.FromArgb(20, 20, 24);
btnPlay.FlatAppearance.MouseOverBackColor = Color.FromArgb(46, 46, 54);
btnPlay.FlatStyle = FlatStyle.Flat;
btnPlay.Font = new Font("Segoe UI", 12F, FontStyle.Bold);
btnPlay.ForeColor = Color.White;
btnPlay.Location = new Point(105, 210);
btnPlay.Margin = new Padding(3, 2, 3, 2);
btnPlay.Name = "btnPlay";
btnPlay.Size = new Size(228, 39);
btnPlay.TabIndex = 0;
btnPlay.Text = "PLAY";
btnPlay.UseVisualStyleBackColor = false;
btnPlay.Click += BtnPlay_Click;
```

Name identifică obiectul în cod, **Text** stabilește inscripția PLAY, iar Size și Location definesc dreptunghiul din container. Font și ForeColor controlează textul. Proprietățile FlatAppearance stabilesc bordura și feedback-ul la hover sau apăsare.

Apăsare PLAY → Click → BtnPlay_Click() → StartNewGame()

Operatorul += abonează metoda BtnPlay_Click la eveniment. Parametrul sender reprezintă obiectul care a produs evenimentul, iar EventArgs e transportă datele generale ale acestuia. Metoda din Form1.cs transmite acțiunea la StartNewGame(), prezentată în Codul 2.

RestartGame() resetează datele, SetPlaying() ascunde meniul și face HUD-ul vizibil, apoi Start() permite actualizările Timer-ului. Un singur Click se transformă astfel într-o rundă nouă, pregătită complet.

De la RESPAWN la o rundă nouă

Al doilea exemplu este **btnRespawn**. Câmpul de tip Button este declarat în Form1.Designer.cs, creat în InitializeComponent() și adăugat în pnlGameOver. Text este RESPAWN, iar Name este btnRespawn. Linia **btnRespawn.Click += BtnRespawn_Click;** conectează butonul la codul manual.

RESPAWN → Click → BtnRespawn_Click() → RestartGame()

Runda imediat după RESPAWN: 1 - HP refăcut; 2 - XP și nivel resetate; 3 - valul 1; 4 - lista de upgrade-uri goală.



Ce face fiecare nivel al aplicației?

Nivel	Responsabilitate în acest exemplu
Designer	Păstrează obiectul Button, proprietățile, părintele și asocierea evenimentului.
Form1.cs	Primește Click prin BtnRespawn_Click și coordonează RestartGame().
GameSession	Golește obiectele rundei, resetează jucătorul, valurile și upgrade-urile.
ScreenController / HUD	Afișează ecranul de joc și valorile inițiale, apoi solicită redesenarea.

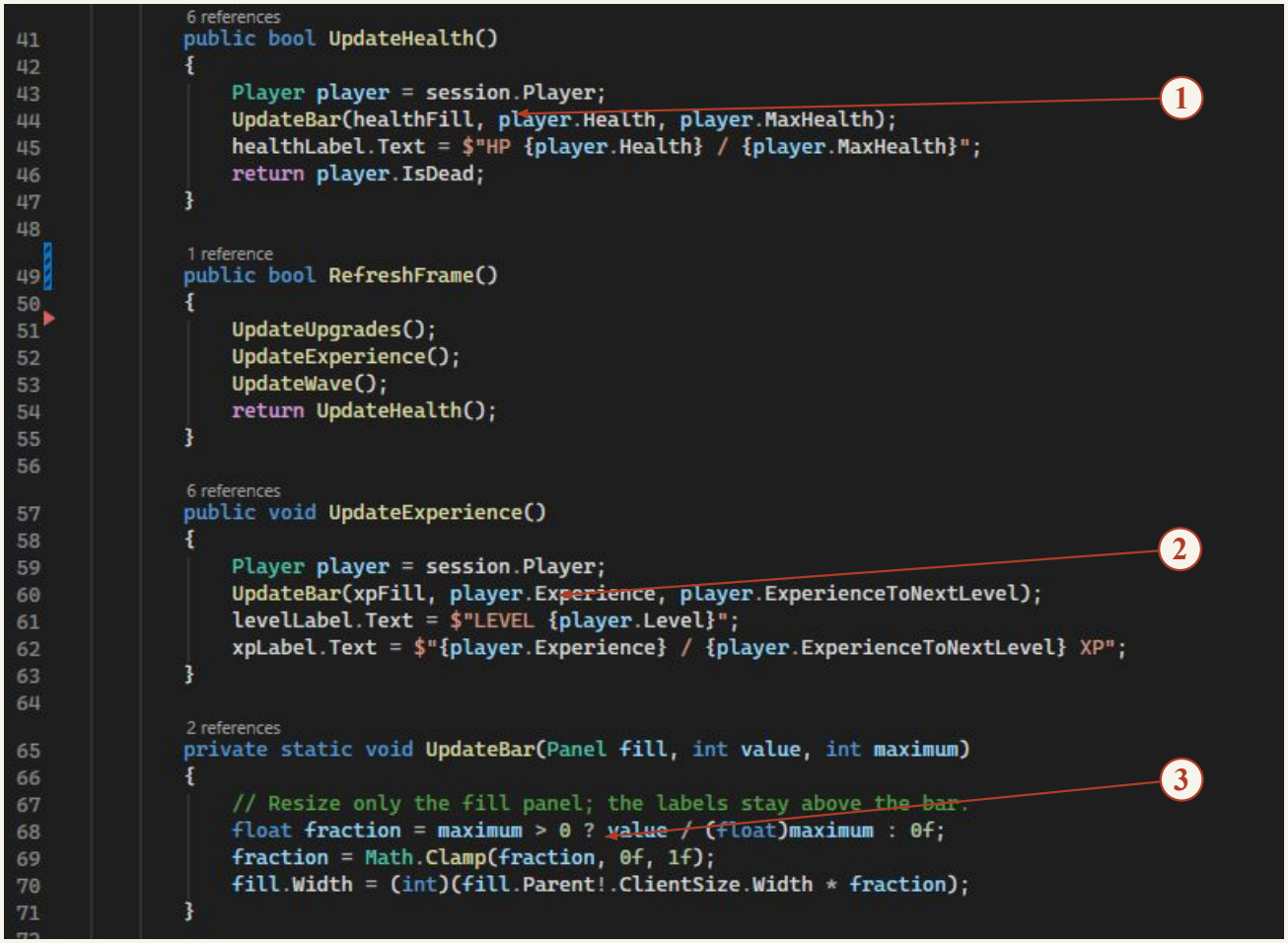
Separarea este utilă când modific interfața: pot schimba inscripția sau poziția butonului în Designer fără să rescriu resetarea sesiunii. Invers, pot completa regulile de resetare în GameSession fără să mut butonul sau să modific resursa grafică a meniului.

Și btnUpgrade1 are o asociere Click în Designer. Metoda sa trimite indicele 0, iar LevelUpController validează alegerea și modifică statisticile. Controlul vizual doar inițiază acțiunea; regulile rămân în clasele corespunzătoare.

Integrarea interfeței cu jocul

HudController primește referințe la controalele create în Designer și la GameSession. Astfel, citește aceleași date ca logica jocului și le prezintă prin Text sau Width. Captura editorului arată metodele reale UpdateHealth(), UpdateExperience() și UpdateBar().

Codul HUD-ului în Visual Studio: 1 - viața; 2 - XP și nivelul; 3 - formula comună a barelor.



Stare	Efect principal asupra interfeței și rundei
MainMenu / Settings	Panoul corespunzător este vizibil; HUD-ul este ascuns.
Playing	Timer-ul actualizează sesiunea; HUD-ul citește valorile noi.
LevelUp / Paused	Panoul este vizibil; Tick returnează înaintea logicii jocului.
GameOver	Panoul Game Over este afișat; runda așteaptă RESPAWN.

La fiecare Tick acceptat, sesiunea primește mouse-ul în coordonate client și direcția tastelor. RefreshFrame() actualizează HUD-ul. Dacă există niveluri în așteptare, se deschide Level Up; dacă jucătorul a murit, se afișează Game Over. Invalidate() solicită un nou OnPaint, fără să deseneze direct în evenimentul butonului.

5. Explicarea codului jocului

5.1 Mișcarea jucătorului

GameInputController păstrează tastele apăsate într-un HashSet<Keys>. D și A dau direcția orizontală, iar S și W direcția verticală. La eliberare, tasta este scoasă din set. Poziția se actualizează în Tick după starea tastelor.

Codul 12. Direcția tastelor WASD

UI/Input.cs:72-81
GameInputController.GetMovementDirection()

```
public PointF GetMovementDirection()
{
    float x = (keysDown.Contains(Keys.D) ? 1 : 0) - (keysDown.Contains(Keys.A) ? 1
    : 0);
    float y = (keysDown.Contains(Keys.S) ? 1 : 0) - (keysDown.Contains(Keys.W) ? 1
    : 0);
    // Normalize diagonals so they move at the same speed as straight lines.
    if (x != 0 && y != 0)
        return new PointF(x * 0.7071f, y * 0.7071f);
    return new PointF(x, y);
}
```

Înmulțirea cu **0.7071f** normalizează diagonala. Player.Move() înmulțește direcția cu Stats.MoveSpeed și o adaugă la X și Y. ClampToArena() limitează poziția la zona client; viteza inițială este 6 unități pe actualizare, înaintea upgrade-urilor.

5.2 Sistemul de shooting

Mouse-ul este convertit prin PointToClient(Cursor.Position), astfel încât țintirea și arena folosesc aceleași coordonate. Player.UpdateAngle() folosește Atan2. CombatSystem.Shoot() calculează gura țevii, direcția către cursor și o normalizează. Dacă mouse-ul este prea aproape, se păstrează sensul armei pentru a evita tragerea inversă.

Codul 13. Lovitura de aproape sau crearea proiectilului

Game/Combat.cs:33-46
CombatSystem.Shoot() • partea finală a metodei

```
// Check the body-to-muzzle path so a touching enemy cannot be skipped.
Enemy? closeEnemy = FindHit(center, muzzle, true);
if (closeEnemy is not null)
    ApplyBulletHit(closeEnemy, dx, dy);
else
    game.Bullets.Add(new Bullet
    {
        X = muzzle.X - Bullet.Size / 2f, Y = muzzle.Y - Bullet.Size / 2f,
        VelocityX = dx * player.Stats.BulletSpeed,
        VelocityY = dy * player.Stats.BulletSpeed,
        Angle = MathF.Atan2(dy, dx) * 180f / MathF.PI
    });
player.ShootCooldownRemainingMs = player.Stats.FireCooldownMs;
player.RegisterShot();
```

Glonțul are viteza inițială 20 și verifică întregul segment parcurs pentru prima coliziune. Damage-ul este 10-14, critica are inițial 2,5% și multiplicator 1,5 sau 2. Cooldown-ul este 210 ms. Fiecare clic stâng încearcă un foc, fără autofoc.

5.3 Inamicii

Cele trei imagini sunt asociate cu EnemyType. Tipurile urmăresc direct jucătorul; diferențele provin din statistici, dimensiune și reacția la lovitură. Tabelul arată valorile de bază, înainte de scalarea vieții și a recompenselor.

Resursă / tip	HP	Viteză	Contact	XP de bază
Zombie_1 / ZombieFast	30	6	10	12-18
Zombie_2 / ZombieHeavy	80	3,6	20	22-32
Vampire_1 / Vampire	110	6,6	30	40-55

Codul 14. Urmărirea jucătorului

Game/Enemies.cs:138-151
EnemyManager.Update() • calculul deplasării unui inamic

```
PointF center = enemy.GetCenter();
float dx = playerCenter.X - center.X;
float dy = playerCenter.Y - center.Y;
float distance = MathF.Sqrt(dx * dx + dy * dy);
enemy.Angle = MathF.Atan2(dy, dx) * 180f / MathF.PI;
enemy.StaggerRemainingMs = Math.Max(0, enemy.StaggerRemainingMs - elapsedMs);
float speed = enemy.Stats.Speed;
if (enemy.StaggerRemainingMs > 0)
    speed *= enemy.Stats.StaggerSpeedFactor;
if (distance > 0.001f)
{
    enemy.X += dx / distance * Math.Min(speed, distance);
    enemy.Y += dy / distance * Math.Min(speed, distance);
}
```

dx și **dy** sunt diferențele către jucător, iar distance este lungimea direcției. Viteza scade temporar la stagger. După deplasare sunt limitate pozițiile și separate corpurile suprapuse. Viața la apariție crește cu 4% pentru fiecare nivel al jucătorului după nivelul 1.

5.4 Sistemul de valuri

WaveManager pornește de la valul 1. Numărul necesar este **3 + (CurrentWave − 1) × 2**, cu o apariție programată la 650 ms. După toate aparițiile și eliminarea inamicilor vii, urmează 2.000 ms de pauză. ZombieHeavy apare de la valul 3, iar Vampire de la valul 10; alegerea ține cont și de indicele apariției.

5.5 XP și Level Up

La eliminare, recompensa este acordată imediat. Enemy.RollExperienceReward() alege XP de bază și aplică +10% pentru fiecare val după primul, cu multiplicator maxim 4. Player.AddExperience(), din Codul 4, păstrează surplusul și mărește pragul cu 8. GameSession pune fiecare nivel câștigat în PendingLevels; interfața cere apoi câte o alegere pentru fiecare. Numai textul plutitor XP este întârziat cu 250 ms, nu recompensa.

Statistici, vindecare și randare

5.6 Upgrade-urile

UpgradeSystem păstrează câte un contor pentru cele șapte tipuri. Fiecare alegere adaugă același bonus. Valorile procentuale sunt aditive; limita efectivă se aplică în PlayerStats, iar lista HUD prezintă totalul stivelor.

Upgrade normal	**Bonus pentru fiecare alegere**
Health Boost	+10 HP maxim și vindecare de până la 10 HP
Speed Boost / Damage Boost	+3% viteză de mișcare / +5% damage
Durability / Critical Training	+3 puncte procentuale rezistență / +1 punct procentual șansă critică
Bullet Speed / Rapid Fire	+5% viteză proiectile / +4% rată de foc

Rezistența are plafonul 60%, iar șansa critică totală 35%. Rapid Fire calculează cooldown-ul prin 210 / (1 + bonusul ratei de foc), fără să scadă direct procentul din timp.

5.7 Sistemul HP și bandajele

Player.TakeDamage() aplică rezistența, păstrează minimum 1 damage și oferă 750 ms de invulnerabilitate după o lovitură. La eliminarea unui inamic, condiția reală este **Random.Shared.NextDouble() < 0.10**. Bandajul vindecă până la 20 HP, rămâne pe teren la viață plină și are un hitbox de 38 × 38 pentru desenul de 28 × 28. Vindecarea produce text verde; resetarea golește bandajele.

5.8 Randarea grafică

OnPaint() transmite Graphics la GameRenderer.Draw(), care desenează terenul, obiectele și personajele. Sprite-urile provin din Resources; bandajul este desenat prin forme. Jucătorul se rotește cu player.Angle în jurul pivotului sprite-ului.

Codul 15. Rotirea și desenarea unui inamic

Game/GameRenderer.cs:86-98
GameRenderer.DrawEnemies() • fragmentul de randare

```
PointF center = enemy.GetCenter();
GraphicsState state = g.Save();
// Rotating around the center keeps the sprite in place while it turns.
g.TranslateTransform(center.X, center.Y);
g.RotateTransform(enemy.Angle + 90f);

RectangleF spriteBounds = new(-size / 2f, -size / 2f, size, size);
ImageAttributes? tint = null;
if (enemy.HitFlashTimer > 0)
    tint = enemy.CriticalHitFlash ? criticalFlashAttributes : hitFlashAttributes;
DrawSprite(g, sprite, spriteBounds, tint);

g.Restore(state);
```

Save() și Restore() izolează transformările. Originea se mută în centrul inamicului, se aplică rotația, apoi DrawSprite desenează imaginea. Renderer-ul reutilizează bitmap-urile din resurse și eliberează obiectele proprii. Invalidate() solicită un nou OnPaint.

6. Reflecție personală

Prin acest proiect am înțeles mai bine diferența dintre a construi o interfață și a programa comportamentul ei. Toolbox și Designer mă ajută să așez controalele, dar rezultatul devine util numai după ce proprietățile și evenimentele sunt conectate corect. Un exemplu simplu este PLAY: aspectul este în Designer, iar pornirea rundei ajunge în metodele din Form1.

Am învățat să folosesc Panel pentru meniuri, Label pentru informații și Button pentru acțiuni. La HP și XP, partea vizuală m-a ajutat să urmăresc valorile din clase. Anchor și recentrarea panourilor arată de ce o interfață nu trebuie gândită doar pentru o singură dimensiune de fereastră.

Una dintre provocări a fost organizarea codului și păstrarea unei interfețe ușor de modificat. Istoricul include refactorizări, iar structura actuală separă sesiunea, lupta, progresia și controlerele UI. Simplificarea funcțiilor și meniul Debug fac mai ușoară urmărirea unei probleme: pot verifica separat valorile, evenimentul și rezultatul afișat.

7. Git și GitHub

7.1 Ce sunt Git și GitHub?

Git urmărește local versiunile fișierelor prin commit-uri. GitHub găzduiește repository-uri și permite accesul la ele prin rețea. În acest proiect, origin indică repository-ul Zombie-Survivors al contului rimikkusu. Git și GitHub au roluri diferite, dar sunt folosite împreună. [6, 7]

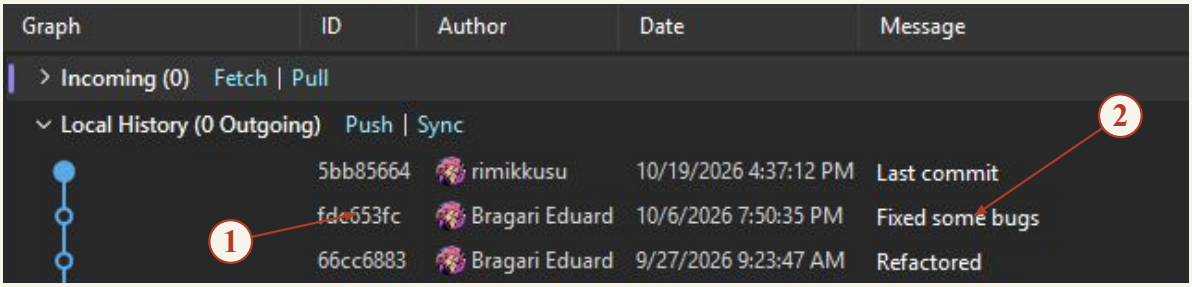
7.2 Utilizarea Git în proiect

Un flux obișnuit este **git status** pentru verificare, **git add .** pentru pregătire, **git commit -m "mesaj"** pentru salvarea versiunii și **git push** pentru trimitere. **git pull** preia și integrează schimbările, iar **git log** arată istoricul. Aceste comenzi sunt explicate, fără să creez sau să trimit un commit pentru documentație.

7.3 Repository-ul proiectului

Commit-uri observate: **5bb8566** - „Last commit”, 19.10.2026; **fdc653f** - „Fixed some bugs”, 06.10.2026; **66cc688** - „Refactored”, 27.09.2026. Versiunea locală documentată are și modificări necomise; nu presupun că toate se află deja pe GitHub.

Istoric real în Visual Studio, ramura main: 1 - identificatori; 2 - mesaje de commit.



8. README și alte comenzi Git

8.1 Ce este README?

README este fișierul de prezentare al unui repository: descrie scopul, cerințele de rulare, instalarea și comenzile utilizatorului. În directorul de lucru analizat, **README.md este șters local**, deci nu prezint un conținut inexistent drept documentație actuală. Pentru o versiune viitoare ar fi utile: descrierea jocului, .NET 10 și Windows, deschiderea soluției, comenzile WASD/mouse/ESC/F3 și structura Game/UI.

8.2 Comenzi Git utile

Comandă	Explicație
git init	Inițializează un repository în folderul curent.
git clone <adresă>	Copiază un repository existent, împreună cu istoricul său.
git status	Afișează fișierele modificate, pregătite sau neurmărite.
git add .	Pregătește modificările din directorul curent, inclusiv ștergeri.
git commit -m "mesaj"	Creează un commit din modificările pregătite.
git push	Trimite commit-urile către repository-ul remote configurat.
git pull	Preia și integrează schimbările din ramura remote urmărită.

Comandă	Explicație
git log / git diff	Arată istoricul / diferențele dintre versiuni sau față de fișierele locale.

9. Concluzie

Realizarea jocului Zombie Survivors m-a ajutat să leg noțiunile de la Programarea vizuală de o aplicație pe care o pot folosi și explica. Am construit meniuri din controale Windows Forms, am configurat proprietăți și am asociat evenimentele cu metode C#. Bara HP, progresul XP și alegerea upgrade-urilor arată direct cum datele din clase ajung în interfață.

Am înțeles că o interfață clară depinde și de organizarea codului. Separarea dintre Designer, formular și clasele auxiliare îmi permite să urmăresc traseul unei acțiuni: apăsarea unui buton, verificarea stării, modificarea sesiunii și actualizarea ecranului. Această organizare este utilă atât când adaug o funcție, cât și când rezolv o eroare.

Consider că proiectul își îndeplinește rolul educațional: include o rundă de joc, progresie și meniuri funcționale într-o aplicație desktop C#. Pentru continuare, aș îmbunătăți documentația repository-ului și aș urmări mai atent comportamentul la diferite dimensiuni de fereastră. Cunoștințele exersate aici pot fi aplicate și la alte aplicații cu formulare, controale și evenimente.

10. Bibliografie

1. Microsoft Learn. Windows Forms overview.
https://learn.microsoft.com/en-us/dotnet/desktop/winforms/overview/

2. Microsoft Learn. Toolbox.
https://learn.microsoft.com/en-us/visualstudio/ide/toolbox?view=visualstudio

3. Microsoft Learn. C# documentation.
https://learn.microsoft.com/en-us/dotnet/csharp/

4. Microsoft Learn. Create a Windows Forms app in Visual Studio.
https://learn.microsoft.com/en-us/dotnet/desktop/winforms/get-started/create-app-visual-studio

5. Microsoft Learn. Events in Windows Forms controls.
https://learn.microsoft.com/en-us/dotnet/desktop/winforms/controls/events

6. Pro Git. Getting Started - About Version Control.
https://git-scm.com/book/en/v2/Getting-Started-About-Version-Control

7. GitHub Docs. About repositories.
https://docs.github.com/en/repositories/creating-and-managing-repositories/about-repositories

8. Bragari Eduard. Zombie Survivors - proiectul analizat.
https://github.com/rimikkusu/Zombie-Survivors

Material de referință pentru structură: PAD_Studiu_Nr1.docx.pdf, furnizat pentru realizarea studiului.